



Home

Explore

20+ Notifications

Chat

Grok

Premium

Articles

Profile

More

Post

OxDipper
@Dipper_pol



← Article



Information Theory on Polymarket: How 77-year old formula prints million for quants

14

82

491

1.1M



Repost

The 77-year-old formula that built the only hedge fund with zero losing years in 20 seasons - and why it still works on prediction markets today.

In 1986, *Barron's* ranked 1,026 mutual funds by performance. Claude Shannon's personal portfolio *outperformed* 1,025 of them.

Shannon wasn't a fund manager. He wasn't a Wall Street quant. He was the MIT professor who invented information theory - the math behind every JPEG, every modem, every compression algorithm on Earth. Between the late 1950s and 1986, his stock portfolio returned **28% annually**. Over the same three decades, Warren Buffett's Berkshire Hathaway returned 27%.

Shannon beat Buffett using the same math he'd used to design telephone lines

This sounds like a coincidence. It isn't. Shannon's approach was methodical, derived directly from his own information theory papers. He wasn't picking stocks by gut. He was measuring the information content of every bet he took - in bits - and sizing positions accordingly.

And one of his colleagues took the method even further.

Tony Zoppi
@TonyZoppi

...

Search

Relevant people

OxDipper
@Dipper_pol
polymarket research
like morning news | @

Live on X

ClaudeDevs is hosting
Code with Claude 2026

Charlie Kirk is hosting
Indiana Victory Lap + The
Women-First Economy + ...

Markets & Mayhem is
Silver Surges -- Are We
Headed Back to \$100?

NEWSMAX is hosting
The Rob Carson Show LIVE
(5/6/2026) | NEWSMAX ...

Mario Nawfal is hosti
LIVE: TRUMP X MELANI
AT MILITARY MOTHER'S D.

Show more

What's happening

News · Trending
Ted Turner
Trending with Cartoon Network

Entertainment · Trending
Elliot Page

Sports · Trending
Kerry Wood

Trending in United States
Legally Blonde

Show more

Terms of Service | Privacy Policy |
Accessibility | Ads info | More ...





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post

Enter Edward Thorp

Edward Thorp - math professor, blackjack legend - walked into Shannon's MIT office in the early 1960s with a card-counting system. Shannon listened, then handed him a Bell Labs paper by John Kelly titled *A New Interpretation of Information Rate*. The paper used Shannon's entropy to derive the mathematically optimal bet size for any game with positive edge.

Thorp took the math to Vegas. Won \$11,000 in a single weekend at blackjack. Then he took it to Wall Street.

So why does this matter for Polymarket?

Most Polymarket traders measure edge in dollars. They track wins, ignore losses, and confuse luck with skill. Shannon and Thorp measured edge in **bits of information** - and built portfolios that compounded for decades while everyone else blew up on a single bad call.

"Bits of information" isn't a metaphor. It's literally how the math works. A Polymarket position has a measurable information-theoretic edge before you place it - expressed as a real number, in bits.

If you know how to compute that number, you know exactly how much alpha you're picking up per trade. If you don't, you're guessing. This article is about three specific tools from information theory that work on Polymarket right now:

1. **Measuring edge in bits** via KL-divergence - before you place the trade.
2. **Fusing contradicting signals** via max-entropy - without overfitting on backtests.
3. **Detecting insiders** via entropy collapse - before the news breaks.

The framework is 77 years old. It beat Vegas, then Wall Street, then bond markets, then options desks. Polymarket is just the newest venue.

By the end of this article, you'll be able to calculate your edge in bits on any open market on Polymarket — the same way Shannon calculated his in stocks and Thorp calculated his in warrants. That number will tell you whether to trade or walk away.

Tony Zoppi  
@TonyZoppi

...





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post

Let's start with the math Shannon used.



What Is Entropy, In Plain English ?

Entropy is a measure of uncertainty. The higher it is, the less you know about the outcome.

Example 1 - The Coin

A fair coin is 50/50. Entropy = 1 bit. That's the maximum for a binary event. An unfair coin 90/10 (almost always heads) has entropy of just 0.47 bits. Less uncertainty.

Example 2 - Polymarket

A "Will X happen?" market at 0.50 has entropy 1 bit. At 0.95 - just 0.29 bits. The closer to 0.50, the more unknown is packed into the market, the more information you extract when the outcome resolves.

Example 3 - Language

Letter **E** appears in 13% of English text, **Z** in 0.07%. English has ~4.1 bits/character. **This is why zip-archives work** - they exploit letter predictability. You can do the same with Polymarket prices: if you predict better than the market, you are literally compressing its uncertainty.

$$H(X) = - \sum p(x_i) \cdot \log_2 p(x_i)$$

X - random variable (the market outcome)

p(x_i) - probability of each outcome

log₂ - log base 2 (that's why the answer is in **bits**)

minus sign - makes the result positive (log of a probability is always negative)

```
python   
  
import numpy as np  
  
def shannon_entropy(p):  
    """Returns entropy in bits for a binary outcome."""  
    if p <= 0 or p >= 1:  
        return 0.0  
    q = 1 - p  
    return -(p * np.log2(p) + q * np.log2(q))  
  
# Fair market (max uncertainty)  
print(shannon_entropy(0.50)) # 1.000 bits  
print(shannon_entropy(0.35)) # 0.934 bits  
print(shannon_entropy(0.95)) # 0.286 bits
```





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post



Shannon entropy $H(p)$ for any binary Polymarket outcome. The curve peaks at exactly 1 bit when $p = 0.5$ - that's the most uncertainty a binary event can hold. Notice the symmetry: a market at 0.35 has the same entropy as one at 0.65 (0.934 bits). Both extremes collapse to 0 - once a market is resolved, there's no information left to extract.

Entropy tells you how uncertain a market is. But how do you measure how much your estimate differs from the market? Enter KL-divergence.

Use-Case I - Measuring Edge via KL-Divergence

Thorp calculated a version of this number on every warrant he traded. Buffett uses it implicitly when sizing positions. On Polymarket, it has a clean name: KL-divergence.

Polymarket has 200+ active markets at any moment. Which ones to trade? Common answer: *"the ones where I have edge."* But how do you **quantify** edge before placing?

$$DKL(P\|Q) = \sum p(xi) \cdot \log_2[p(xi)/q(xi)]$$

P - your probability estimate

Q - the market's estimate (the price)

DKL - literally **how many bits of info you have that the market doesn't**

KL-divergence isn't a metaphor for edge. It literally is edge - measured in bits.

Market: *"Will Trump pardon X by Dec 31?"*. Current price: **0.35** (market thinks 35% YES).

Your model (based on 20 historical pardons): **0.55**.

Calculation:

$$\begin{aligned}DKL &= 0.55 \cdot \log_2(0.55 / 0.35) + 0.45 \cdot \log_2(0.45 / 0.65) \\&= 0.55 \cdot 0.652 + 0.45 \cdot (-0.530) \\&= 0.359 - 0.239 \\&= \mathbf{0.120 \text{ bits}}\end{aligned}$$

- 0.00 bits - No edge. Skip.

Tony Zoppi  
@TonyZoppi

...





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post

- 0.05 bits - Weak. Trap zone. Fees win.
- 0.10-0.20 - Real edge. Pro zone. Trade.
- > 0.30 - You're wrong. Check your math.

```
python

import numpy as np

def kl_divergence_binary(p_mine, p_market):
    """Returns KL divergence in bits for a binary market.
    p_mine - your estimate (0..1)
    p_market - market price (0..1)
    """
    # Clip to avoid log(0)
    p_mine = np.clip(p_mine, 0.01, 0.99)
    p_market = np.clip(p_market, 0.01, 0.99)

    q, qm = 1 - p_mine, 1 - p_market
    return (p_mine * np.log2(p_mine / p_market)
            + q * np.log2(q / qm))

# Your estimate 0.55, market price 0.35
edge = kl_divergence_binary(0.55, 0.35)
print(f"Edge: {edge:.3f} bits") # 0.120 bits
```

Workflow

1. Pull all open markets via Polymarket API (or scrape).
2. Compute your probability estimate for each
3. Calculate D_{KL} for every market.
4. Sort descending by edge.
5. Trade top 5–10 only. Ignore the rest.

Common mistakes

- KL is **asymmetric**: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. First arg is always your estimate.
- Breaks on $p=0$ or $p=1$. Always clip to $[0.01, 0.99]$.
- High $D_{KL} \neq$ guaranteed profit. Fees, liquidity, slippage still matter.

You can measure edge from a single signal. But what if you have five signals that contradict each other?

Use-Case II - Max-Entropy Signal Fusion

You have several signals: news, a historical base rate, the order book, Twitter sentiment, other models' predictions. They contradict. How to combine?

Why naive approaches fail?

- **Averaging** - throws away information, weights all signals equally even if one dominates.





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post

- **ML ensembles** - need tons of data, overfit on backtests, die in live trading.
- **Manual weights** - subjective, collapse in new market regimes.

When you have incomplete information, the only honest probability distribution is the one with maximum entropy under the known constraints.

Don't sneak in hidden assumptions. Use only the facts you actually have. Leave everything else maximally uncertain.

python



```
import numpy as np
from scipy.optimize import minimize

def max_entropy_fusion(signals, confidences):
    """
    signals      - list of probability estimates [p1, p2, ..., pN]
    confidences   - list of weights (higher = more trust)
    Returns a single fused probability via max-entropy weighting.
    """
    signals      = np.array(signals)
    confidences   = np.array(confidences)

    # Constraint: weighted mean matches signal consensus
    target = np.sum(signals * confidences) / np.sum(confidences)

    # Max-entropy under mean constraint on {0,1} = Bernoulli(target)
    return np.clip(target, 0.01, 0.99)

signals      = [0.40, 0.60, 0.50]
confidences   = [1.0, 2.0, 1.5] # news most trusted

fused = max_entropy_fusion(signals, confidences)
print(f"Fused probability: {fused:.3f}") # ~0.522
```

Three signals on a single market:

- **Signal 1** (base model): $p \approx 0.40$
- **Signal 2** (news scoring): $p \approx 0.60$
- **Signal 3** (order book): $p \approx 0.50$

Naive average: 0.50. Max-entropy, weighted by signal confidence (variance), gives a single distribution that satisfies all constraints and adds nothing else. No overfitting - because there's *nothing to overfit to*.

Max-entropy doesn't overfit. It *can't* - by construction. You cannot extract more information from a distribution than the constraints you fed it. Jaynes proved this in 1957.





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post



You can measure edge and fuse signals. One question left: how do you know if someone on the other side knows more than you?

Use-Case III - The Insider Detector

Polymarket has insiders. Political markets - people know decisions before the press release. Sports - staff knows injuries. Person-specific markets - friends know plans. Can you detect them moving?

A normal market drifts slowly, reacting to small news flow. An insider moves differently: they know the outcome, open a large position, the price snaps in one direction, and **entropy collapses sharply**.

Build a time series of market entropy and watch its derivative. Insider signals show up *before* public news.

$$\text{alert if } |dH/dt| > k \cdot \sigma H$$

dH/dt - rate of change of entropy over time

σ_H - historical std deviation of entropy changes

k - threshold (typically 3 - 3-sigma rule)

Method, step-by-step

- Compute market entropy $H(t)$ every 5 minutes using current price.
- Build time series of $H(t)$.
- Compute rolling derivative dH/dt .
- Alert when $|dH/dt|$ exceeds 3 standard deviations from historical mean.
- Check: was there public news at that moment? If no - **someone knows**.

Tony Zoppi  
@TonyZoppi

...





Home

Explore

20+ Notifications

Chat

Grok

Premium

Articles

Profile

More

Post

python

```
import numpy as np
import pandas as pd
```

```
def entropy(p):
    p = np.clip(p, 0.01, 0.99)
    return -(p * np.log2(p) + (1-p) * np.log2(1-p))

def detect_insiders(prices_df, window=12, k=3.0):
    """
    prices_df : DataFrame with columns ['timestamp', 'price']
                (5-minute intervals recommended)
    window    : rolling window for baseline stats
    k          : sigma threshold
    """
    prices_df['H'] = prices_df['price'].apply(entropy)
    prices_df['dH'] = prices_df['H'].diff()
    prices_df['sigma'] = prices_df['dH'].rolling(window).std()
    prices_df['alert'] = np.abs(prices_df['dH']) > k * prices_df['sigma']

    return prices_df[prices_df['alert']]

# Returns rows where entropy collapsed abnormally fast
alerts = detect_insiders(market_prices)
print(f"Detected {len(alerts)} anomalies")
```

Oct 13, 2024 - market *"Will Trump win Pennsylvania?"*. Between 14:00 and 14:15 EST, price moves from 0.52 to 0.68 on large orders from a single wallet. Entropy drops from 0.998 to 0.905 bits in 15 minutes - a 7-sigma event. No public news at that moment. Next public news arrives 47 minutes later. Wallet nets ~\$340k during that hour.

The SEC uses similar techniques to detect insider trading in equities - academic literature dates back to early 2000s. Almost nobody applies it to Polymarket, despite public data and *more* insiders.

Everyone watches price. Quants watch entropy. It tells them an insider arrived - an hour before the news does.

Tony Zoppi  
@TonyZoppi

...





Home



Explore



20+

Notifications



Chat



Grok



Premium



Articles



Profile



More

Post

The Closer

\$0 → measurable edge in 5 lines of Python. Three tools. One framework. 77 years old.

- **Edge in bits.** KL-divergence quantifies your alpha before you place. If $D_{KL} < 0.05$, you're guessing. If $D_{KL} > 0.10$, you're trading. If $D_{KL} > 0.30$, your math is wrong.
- **No overfitting.** Max-entropy fuses signals without inventing information. Jaynes proved it in 1957. ML ensembles still haven't

7:56 AM · May 4, 2026 · 11M Views
caught up. 14 82 491 1.5K
Relevant **Insiders, not news.** Entropy collapses minutes before the press release. Watch dU/dt or stay surprised. View quotes >

Post your reply

Reply





 Home

 Explore

20+
 Notifications

 Chat

 Grok

 Premium

 Articles

 Profile

 More

Post



Tony Zoppi  
@TonyZoppi

...





 Home

 Explore

20+
 Notifications

 Chat

 Grok

 Premium

 Articles

 Profile

 More

Post



Tony Zoppi  
@TonyZoppi

...

